Trabajo Práctico Unidad 6

# Mantenimiento, respaldo y recuperación de VoltTech

Informe de ejecución dividido por tareas · PostgreSQL 17.10 · Base volttech_u6

| | | | |
|---|---|---|---|
| **Alumno** | Joaquin Antonio Mendez Reynoso | **Materia** | Bases de Datos II · Comisión 4 |
| **Motor** | PostgreSQL 17.10 (localhost:5432) | **Base principal** | volttech_u6 |
| **Repositorio** | https://github.com/VIKROOM/db_Mendez_COM4_U6.git | **Fecha** | 10 de noviembre de 2026 |

## Resumen ejecutivo

El trabajo se ejecutó de punta a punta sobre una instancia real de PostgreSQL, usando como base la aplicación VoltTech del equipo (comercio de electrónica) con datos en volumen: 50.002 productos, 20.001 clientes, 200.001 pedidos y 800.002 líneas de pedido. Se partió del análisis de la consigna y de una especificación escrita, se preparó el entorno con roles de mínimo privilegio, se respaldó la base con verificación de integridad, se monitoreó su salud, se ejecutó un mantenimiento justificado por métricas y se realizó un simulacro de desastre completo con medición de RPO y RTO. Todos los resultados de este informe salen de archivos de evidencia generados por las mismas corridas: no hay valores estimados ni copiados de una corrida ideal.

## Mapa de tareas y resultado

| # | Tarea | Entregable principal | Resultado |
|---|---|---|---|
| 1 | Análisis de la consigna y especificación | SPEC_U6.md | Plan y reglas definidos |
| 2 | Preparación del entorno, roles y datos | soporte/00 y 01, pgpass.conf | 3 roles + bitácora + vista |
| 3 | Respaldo lógico con control de falla | backup_volttech.ps1 | 10,81 MB · 58 objetos · SHA-256 |
| 4 | Monitoreo de salud | health_check.sql | Antes y después comparables |
| 5 | Mantenimiento justificado | maintenance.sql | 5000 tuplas muertas recuperadas |
| 6 | Simulacro de desastre (RPO/RTO) | restore_test.ps1 | RTO 7,60 s · RPO 58 s · OK |
| 7 | Informe, autoevaluación y publicación | informe_u6.pdf · verificación | 102 PASS / 0 FAIL |

| 7,60 s | 58 s | 7/7 | 102 / 0 |
|---|---|---|---|
| RTO observado (meta ≤ 20 min) | RPO observado (meta ≤ 24 h) | Conteos validados contra el snapshot | Autoevaluación PASS / FAIL |

Cada tarea se presenta con el mismo esquema: consigna, qué se hizo, resultado obtenido, problema real encontrado con su corrección y evidencias auditables. Los anexos finales concentran el índice de archivos y los comandos de reproducción.

## 1 Análisis de la consigna y especificación — COMPLETADA

**Consigna.** Interpretar el escenario, definir el alcance, la frecuencia de cada tarea, las metas de RPO/RTO y las reglas de seguridad no negociables antes de escribir código.

**Qué se hizo.**

- Se redactó **SPEC_U6.md** (37 líneas) con alcance explícito: respaldo lógico diario, chequeo de salud, revisión semanal de mantenimiento y restauración de prueba; quedan fuera el respaldo físico, PITR y replicación.
- Se definió una tabla de tareas con frecuencia, herramienta, identidad, condición de éxito y evidencia verificable para cada una (diaria 02:00, diaria al inicio de jornada, semanal, semanal).
- Se fijaron las metas: **RPO ≤ 24 h** y **RTO ≤ 20 min**, medidas con datos reales y no con estimaciones.
- Se escribieron 7 reglas de seguridad: no tocar volttech_u6, operaciones destructivas solo en **volttech_u6_restore_test**, contraseñas únicamente en pgpass.conf, mínimo privilegio, prohibición de **VACUUM FULL** periódico, aborto ante el primer error y respaldo válido solo si se restaura y valida.

La especificación funciona como contrato: la autoevaluación comprueba automáticamente que el plan mencione las frecuencias, RPO/RTO, la base descartable, pgpass.conf y la prohibición de VACUUM FULL.

**Evidencias:** SPEC_U6.md · evidencias/verificacion_entrega.txt (controles de plan).

## 2 Preparación del entorno, roles y datos — COMPLETADA

**Consigna.** Dejar una instancia reproducible: base poblada, identidades de mínimo privilegio, bitácora de operaciones, compatibilidad de nombres y credenciales fuera del repositorio.

**Qué se hizo.**

- Se cargó **volttech_u6** desde los scripts de la app VoltTech del equipo: 2 categorías, 20.001 clientes, 50.002 productos, 200.001 pedidos y 800.002 líneas de pedido, con sus vistas, vista materializada, procedimientos, extensión pg_trgm e índices.
- Se crearon tres roles de mínimo privilegio y el esquema **mantenimiento.bitacora** para dejar registro de cada respaldo, mantenimiento y restauración con fecha, operación, resultado y código de salida.
- El modelo real del equipo usa **linea_pedido** y la consigna nombra **detalle_pedido**. Se creó la vista de compatibilidad **public.detalle_pedido** sobre la tabla real, sin alterar el DDL del proyecto.
- Las contraseñas de los roles quedaron solo en **%APPDATA%\postgresql\pgpass.conf**; los scripts usan **pg_dump -w** y nunca piden ni imprimen credenciales.

| Rol | Privilegios | Uso y límite |
|---|---|---|
| mant_backup | pg_read_all_data + INSERT en mantenimiento.bitacora | Respaldo y snapshot de conteos. Sin escritura sobre datos de negocio. |
| mant_monitor | pg_monitor | health_check.sql: estadísticas, actividad y bloqueos sin acceder a datos. |
| mant_restore | CREATEDB + lectura; CONNECTION LIMIT 10 | Simulacro: crea volttech_u6_restore_test y restaura con -j 4. |

La preparación es idempotente: los scripts de roles y de entorno pueden volver a ejecutarse sin duplicar objetos ni romper permisos.

**Decisión de diseño** Se optó por una vista de compatibilidad en lugar de renombrar la tabla del modelo integrador. Así las validaciones del enunciado funcionan literalmente con detalle_pedido y el esquema de VoltTech sigue intacto.

**Evidencias:** soporte/00_preparacion_entorno.sql · soporte/01_roles_mantenimiento.sql · soporte/setup_pgpass.ps1 · evidencias/counts_pre_backup.txt.

## 3 Respaldo lógico con verificación de integridad — COMPLETADA

**Consigna.** Generar un respaldo restaurable, fechado, con log, sin contraseñas embebidas y con salida distinta de cero ante cualquier falla.

**Qué se hizo.**

- El script toma un **snapshot de conteos antes** del respaldo para poder comparar después de restaurar; ese snapshot es la referencia objetiva del RPO lógico.
- Ejecuta **pg_dump -Fc -Z 6 -w** con el rol **mant_backup** y escribe primero un archivo **.parcial**.
- Ante error: aborta con código distinto de cero, elimina el **.parcial** y registra **ERROR** en el log y en la bitácora. Nunca deja un archivo incompleto con nombre válido.
- Si termina bien: verifica el dump con **pg_restore --list**, lo renombra con fecha y hora, calcula **SHA-256** y aplica retención solo a sus propios archivos (patrón volttech_u6_*.dump).

| Control | Valor observado |
|---|---|
| Archivo | volttech_u6_20261110_203628.dump (formato custom -Fc, compresión -Z 6) |
| Tamaño y objetos | 11.332.222 bytes (10,81 MB) · 58 entradas en el TOC |
| Duración | 2,9 (20:36:28–20:36:31) |
| Integridad | SHA-256 621D798175B2077F…64E92C13 |
| Bitácora | Fila OK registrada en mantenimiento.bitacora |
| Retención | 14 días; 0 archivos eliminados en esta corrida |

**Prueba de falla ejecutada.** Se forzó un error con `-PgDumpExtra '--table=public.tabla_que_no_existe'`: pg_dump terminó con **exit ?**, el log quedó en **ERROR**, se eliminó el .parcial y no apareció ningún .dump válido.

**Problema real y corrección**

El respaldo terminó bien en la primera ejecución porque los errores típicos se cubrieron en el diseño: el INSERT en bitácora se escribe mediante un archivo temporal con psql -f (psql no interpola :'variables' en -c), pg_dump corre con -w y pgpass.conf (sin contraseñas embebidas ni esperas en tareas programadas) y el dump se graba primero como .parcial y se renombra recién tras pasar pg_restore --list. La prueba de falla confirmó ese camino de error: exit 1, .parcial eliminado y ERROR en bitácora.

**Evidencias:** evidencias/backup_ejecucion.log · evidencias/backup_listado.txt · evidencias/backup_sha256.txt · evidencias/fallo_pg_dump.log · logs/.

## 4 Monitoreo de salud de la base — COMPLETADA

**Consigna.** Contar con al menos cuatro controles que permitan detectar degradación antes de que se convierta en incidente.

**Qué se hizo.**

- **Tamaño:** tamaño de la base y de cada tabla, separando tabla, índices y filas vivas.
- **Actividad:** conexiones actuales contra max_connections, commits y rollbacks, cache hit, deadlocks y uso de archivos temporales.
- **Estado de tablas:** tuplas vivas y muertas, porcentaje de muertas, fechas de vacuum y analyze, contadores de autovacuum y uso de índices.
- **Consultas y bloqueos:** sesiones activas, idle in transaction, bloqueos y edad del XID; además, antigüedad del último respaldo OK para vigilar el RPO antes del incidente.

| Indicador | Antes del mantenimiento | Después del mantenimiento |
|---|---|---|
| Tuplas vivas en producto | 50002 | 50002 |
| Tuplas muertas en producto | 5000 | 0 |
| Porcentaje de tuplas muertas | 9,09 % | 0,00 % |
| last_vacuum | (null) | 2026-11-10 20:35:44.130101-03 |
| last_analyze | 2026-11-10 20:22:47.085465-03 | 2026-11-10 20:35:44.523254-03 |
| Tamaño total de producto | 14 MB | 14 MB (sin reescritura) |

**Evidencias:** health_check.sql · evidencias/health_check_antes.txt · evidencias/health_check_despues.txt.

## 5 Mantenimiento justificado por métricas — COMPLETADA

**Consigna.** Observar autovacuum y tuplas muertas, decidir con un umbral documentado y ejecutar solo la acción segura.

**Qué se hizo.**

- Primero se observan los parámetros de autovacuum (activo, scale factor 0,2/0,1, threshold 50, hasta 3 workers, naptime 60 s), los overrides por tabla, las transacciones largas y las estadísticas de cada tabla.
- Umbral documentado: se interviene si una tabla tiene **≥ 5 % de tuplas muertas y ≥ 1.000 filas**, o **≥ 10.000 tuplas muertas**. Solo **producto** calificó, con 5000 muertas sobre 50002 (9,09 %).
- Se ejecutó **VACUUM (ANALYZE, VERBOSE)** únicamente sobre esa tabla mediante \gexec: quedaron 0 tuplas muertas y las estadísticas actualizadas (last_analyze 2026-11-10 20:35:44.523254-03).
- No se usó **VACUUM FULL** en ningún momento: bloquearía la tabla y la consigna lo prohíbe como mantenimiento periódico. La acción y su resultado quedaron en la bitácora.

Lectura del resultado: el espacio no se devuelve al sistema operativo, pero queda disponible para reutilización dentro de la tabla. Eso es exactamente lo que se busca en un mantenimiento rutinario: recuperar espacio interno sin bloquear la operación.

**Corrección de criterio** — Un VACUUM FULL “para recuperar espacio” fue descartado desde el diseño: reescribe la tabla y toma ACCESS EXCLUSIVE, algo prohibido como mantenimiento periódico por la consigna. Se reemplazó por observación previa y VACUUM (ANALYZE) condicionado.

**Evidencias:** maintenance.sql · evidencias/maintenance_salida.txt · evidencias/health_check_antes.txt y _despues.txt.

## 6 Simulacro de desastre con RPO y RTO medidos — COMPLETADA

**Consigna.** Restaurar el respaldo en una base descartable, medir el tiempo real y demostrar que los datos y las reglas lógicas sobrevivieron.

**Qué se hizo.**

- El script valida el dump con **pg_restore --list**, verifica el **SHA-256** y ejecuta un **preflight** de conexiones antes de crear la base. Aborta si el destino es volttech_u6 o no termina en **_restore_test**.
- Arranca el cronómetro inmediatamente antes de **createdb** y restaura con **pg_restore --no-owner --no-privileges --exit-on-error -j 4** usando **mant_restore** (CREATEDB, sin superusuario).
- Detiene el cronómetro al terminar la última validación. El RPO se calcula como la antigüedad del .dump respecto del inicio del simulacro.
- volttech_u6 nunca se toca: el simulacro corre de principio a fin sobre **volttech_u6_restore_test**, que queda conservada para inspección junto con la vista materializada repoblada.

| Métrica / control | Previsto | Observado | Cumple |
|---|---|---|---|
| RPO (pérdida máxima) | ≤ 24 h (86.400 s) | 58 s (0 min 58 s) de antigüedad | Sí |
| RTO (recuperación) | ≤ 20 min (1.200 s) | 7,60 s | Sí |
| Conteos vs. snapshot | 7/7 coincidentes | 7/7 coincidentes | Sí |
| Productos / detalles inválidos | 0 / 0 | 0 / 0 | Sí |
| Huérfanos referenciales | 0 | 0 | Sí |
| Restricciones restauradas | 16 | 16 | Sí |
| Vista detalle_pedido | Presente | 800.002 filas | Sí |

**Conteos validados:** 2 categorías, 20.001 clientes, 50.002 productos, 200.001 pedidos, 800.002 líneas, suma de cantidades 1.599.736 e importe 800.285.374.188,00.

**Objetos de la app restaurados:** vistas v_productos_vigentes, v_pedidos_cliente y v_detalle_pedido_producto; vista materializada mv_facturacion_categoria_mes repoblada con REFRESH; procedimientos registrar_pedido y ajustar_stock; extensión pg_trgm y su índice gin sobre producto.nombre.

**Problema real y corrección**

pg_dump restaura las vistas materializadas sin datos: mv_facturacion_categoria_mes quedaba vacía en volttech_u6_restore_test tras un pg_restore impecable (0 errores y 7/7 conteos). No era un fallo del respaldo sino un comportamiento estándar que el procedimiento debía cubrir: validando solo conteos de tablas, la base habría pasado el chequeo con la vista de reportes vacía. Se agregó REFRESH MATERIALIZED VIEW a soporte/validaciones_restauracion.sql y quedó como evidencia: 26 filas tras repoblar (restore_validaciones.txt, sección 8).

**Evidencias:** evidencias/restore_ejecucion.log · evidencias/restore_metrics.json · evidencias/restore_validaciones.txt.

## 7 Informe, autoevaluación y publicación COMPLETADA

**Consigna.** Documentar el procedimiento y los resultados, verificar la entrega contra la consigna y publicar todo en el repositorio del equipo.

**Qué se hizo.**

- Se redactó **informe_u6.pdf**, de una página, con respaldo utilizado, RPO/RTO previstos y observados, validación, problema real, mejora concreta y uso de IA.
- El uso de OpenCode quedó registrado en BITACORA_IA.md: 12 decisiones de la IA revisadas por el operador, incluidas las propuestas descartadas y por qué (credenciales reemplazadas por pgpass, rechazo de VACUUM FULL, vista de compatibilidad detalle_pedido, límite de conexiones para pg_restore -j 4 y repoblado de la vista materializada).
- Se ejecutó **soporte/verificar_entrega.ps1** contra la entrega completa: **102 PASS / 0 FAIL**, incluyendo archivos, ausencia de contraseñas, ausencia de VACUUM FULL, evidencias y métricas del simulacro.
- El paquete quedó publicado en github.com/VIKROOM/db_Mendez_COM4_U6 (rama main): los logs de cada corrida se versionan como evidencia, .gitignore solo excluye backups/ y __pycache__, y ninguna credencial pasó por el repositorio (pgpass.conf vive en %APPDATA%).

| Elemento | Contenido |
|---|---|
| Raíz del TP | SPEC_U6.md, 4 scripts, informe_u6.md/.pdf, BITACORA_IA.md, README.md, .gitignore |
| soporte/ | preparación de entorno, roles, simulación de actividad, conteos, validaciones y autoevaluación |
| informe/ | generadores Python (reportlab) de los tres informes PDF |
| evidencias/ | 13 archivos de salida cruda de la corrida (logs, conteos, métricas, verificación) |
| logs/ | logs fechados de cada corrida (versionados como evidencia) |
| backups/ | dumps con su .sha256 (regenerables, ignorados por git) |

**Repositorio:** https://github.com/VIKROOM/db_Mendez_COM4_U6.git

**Informe de una página:** Mantenimiento/informe_u6.pdf

**Este informe por tareas:** Mantenimiento/informe_u6_tareas.pdf

**Evidencias:** informe_u6.pdf · BITACORA_IA.md · evidencias/verificacion_entrega.txt.

**Conclusión.** El procedimiento quedó completo y verificable: respaldo con integridad comprobada, prueba de falla exitosa, monitoreo con seis bloques de control, mantenimiento decidido por métricas, restauración real en 7,60 s y validación 7/7 con cero inconsistencias. La base principal volttech_u6 nunca se tocó y ninguna credencial quedó en el repositorio. El siguiente paso natural es programar el respaldo diario y el chequeo de salud con el Programador de tareas de Windows, y alertar cuando el último respaldo OK supere las 20 h.

## Anexo A · Índice de evidencias

Cada resultado del informe se puede auditar en el archivo correspondiente dentro de la entrega.

| Archivo | Qué demuestra |
|---|---|
| evidencias/backup_ejecucion.log | Respaldo OK: 11.332.222 bytes, 58 objetos, SHA-256 y bitácora. |
| evidencias/backup_listado.txt | El dump se puede leer con pg_restore --list. |
| evidencias/backup_sha256.txt | Huella SHA-256 del archivo de respaldo. |
| evidencias/fallo_pg_dump.log | Prueba de falla: exit ?, ERROR y sin .dump válido. |
| evidencias/health_check_antes.txt | Estado previo: producto con 5000 tuplas muertas (9,09 %). |
| evidencias/health_check_despues.txt | Estado posterior: 0 tuplas muertas y ANALYZE actualizado. |
| evidencias/maintenance_salida.txt | Observación de autovacuum y VACUUM (ANALYZE) justificado. |
| evidencias/restore_ejecucion.log | Simulacro completo con preflight y RTO/RPO observados. |
| evidencias/restore_metrics.json | Métricas finales: RTO 7,60 s, RPO 58 s, validaciones OK. |
| evidencias/restore_validaciones.txt | 7/7 conteos, invalid=0, huérfanos=0, 16 restricciones y objetos VoltTech. |
| evidencias/counts_pre_backup.txt | Snapshot de conteos previo al respaldo. |
| evidencias/verificacion_entrega.txt | Autoevaluación: 102 PASS / 0 FAIL. |

## Anexo B · Reproducción

Los comandos se ejecutan desde la raíz de la carpeta del TP. Requieren PostgreSQL 16+ y pgpass.conf configurado.

| Paso | Comando |
|---|---|
| Respaldo diario | `.\backup_volttech.ps1` |
| Salud antes/después | `psql -h localhost -U mant_monitor -d volttech_u6 -X -f .\health_check.sql` |
| Mantenimiento | `psql -h localhost -U postgres -d volttech_u6 -X -f .\maintenance.sql` |
| Simulacro | `.\restore_test.ps1 -Force` |
| Autoevaluación | `.\soporte\verificar_entrega.ps1` |

Repositorio: https://github.com/VIKROOM/db_Mendez_COM4_U6.git · Informe oficial de la Parte 5: informe_u6.pdf (1 página).